

DSA

Module 9

LinkedLists

Author
Srinivas Dande





Java Learning Center

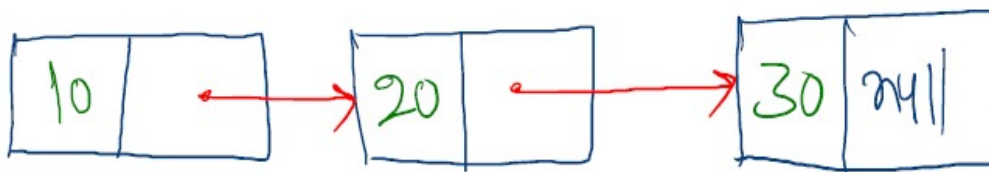
No.1 In Java Training & placement



9. Introduction to LinkedLists

- ◆ Linked List is Linear Data Structure.
- ◆ Linked List is Used to store collection of elements.
- ◆ LinkedLists elements can be accessed in sequential order only.
- ◆ LinkedLists elements will be stored with Node representation.

Ex:



- ◆ Linked list has the following properties.
 - ✓ Successive elements are connected by pointers
 - ✓ The last element points to NULL
 - ✓ Can grow or shrink in size during execution of a program
 - ✓ Does not waste memory space. It allocates memory as list grows but takes some extra memory for pointers.
- ◆ Types of Linked Lists
 - ✓ Singly Linked Lists
 - ✓ Doubly Linked Lists
 - ✓ Circular Linked Lists

9.1 Why LinkedLists

- ◆ There are many other data structures that do the same thing as linked lists.
- ◆ Before discussing linked lists it is important to understand the difference between linked lists and arrays.
- ◆ Both linked lists and arrays are used to store collections of data, and since both are used for the same purpose, we need to differentiate their usage.
- ◆ That means in which cases arrays are suitable and in which cases linked lists are suitable.

Advantages of Arrays:

- ♦ Arrays are Used to store collection of elements of Same type
- ♦ Arrays elements are stored in contiguous locations
- ♦ Arrays elements will be stored with index representaiton.
- ♦ Arrays allow Random access to elements.
- ♦ Search Operation is faster because we can apply Binary Search.
- ♦ Arrays are Cache Friendly.

DisAdvantages of Arrays:

- ♦ Arrays are fixed size.
It means Once you create the array, you can not increse the size.
- ♦ Momory waste when you allocate more space and store less elements.
- ♦ Momory shortage when you allocate less space and store more elements.
You need to resize the array when you want to store more elements.
Resizing is an expensive again.
- ♦ Insert Operation is expensive because it takes time to shift elements
- ♦ Delete Operation is expensive because it takes time to shift elements

Advantages of LinkedLists:

- ♦ LinkedLists are Used to store collection of elements
- ♦ LinkedLists elements will not be stored in contiguous locations
- ♦ LinkedLists elements will be stored with Node representaiton.
- ♦ LinkedLists allow Only sequential acess to elements, No Random access.
- ♦ Insert Operation is faster
- ♦ Delete Operation is faster
- ♦ LinkedLists are dynamic. i.e No Node Creation inadvance

DisAdvantages of LinkedLists:

- ♦ No Random access.
- ♦ Search Operation is expensive
we can not apply Binary Search on LinkedList because of Sequential Access.
- ♦ Momoeiy wastage for Node addresses.
- ♦ LinkedLists are Not Cache Friendly.

Comparison of Linked Lists with Arrays & Dynamic Arrays

<u>Parameter</u>	<u>Arrays</u>	<u>ArrayList</u>	<u>LinkedList</u>
Get Element	$O(1)$	$O(1)$	$O(n)$
Insert at Beginning	$O(n)$	$O(n)$	$O(1)$
Delete at Beginning	$O(n)$	$O(n)$	$O(1)$
Insert at End	$O(1)$	$O(1)$	$O(n)$
Delete at End	$O(1)$	$O(1)$	$O(n)$
Insert at Middle	$O(n)$	$O(n)$	$O(n)$
Delete at Middle	$O(n)$	$O(n)$	$O(n)$

Use-cases:

1. I am writing some programs which has to store very large number of elements. I may not get the contiguous memory because memory may be available as chunks. Which one to choose to store large number of elements.

Ans: LinkedList

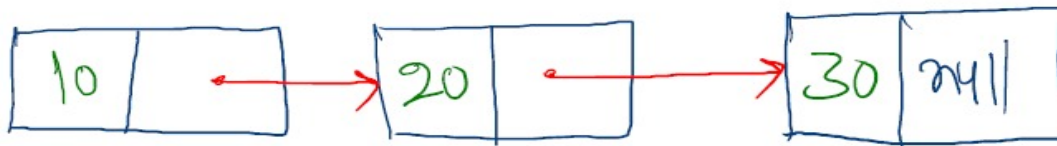
2. Stacks and Queues can be implemented with Arrays and LinkedLists.
Ex: I want to Implement Round Robin Scheduling.

Ans: Use Queue with LinkedLists

9.2. Singly Linked List

- ♦ Singly Linked List consists of a number of nodes
- ♦ Each Node has will have 2 parts
 - a) Data
 - b) Address of Next Node
- ♦ Address of the last node in the list is NULL, which indicates the end of the list.

Ex:



- ♦ Following is a type declaration for a linked list:

```
public class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

- ♦ Main Linked Lists Operations
 - ✓ Traverse: Access the elements in the list
 - ✓ Count: returns the number of elements in the list
 - ✓ Find nth node from the end of the list
 - ✓ Insert: inserts an element into the list
 - ✓ Delete: removes the specified position element from the list

9.2.1. Traverse the LinkedList using Iterative Style

Lab1.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {

    static void displayList(Node headNode) {

        Node currentNode = headNode;

        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);

    }
}
```

9.2.2. Traverse the LinkedList using Recursive Style

Lab2.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {

    static void displayList(Node currentNode) {

        //Base Condition
        if(currentNode==null)
            return ;

        System.out.print(currentNode.data+"\t");
        displayList(currentNode.next); //Recursive Call

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);
    }
}
```



9.2.3. Count the No. of Nodes in the LinkedList

Lab3.java

```
package com.jlccindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    static int length(Node headNode) {

        int length = 0;
        Node currentNode = headNode;
        while (currentNode != null) {
            length++;
            currentNode = currentNode.next;
        }
        return length;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        int len = length(head);
        System.out.println(len);

        Node myhead = null;
        int mylen = length(myhead);
        System.out.println(mylen);
    }
}
```

9.2.4. Insert the Node at the begining of LinkedList

Lab4.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);
        temp.next = headNode;
        return temp;
    }
    static void displayList(Node headNode) {
        Node currentNode = headNode;
        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }
        System.out.println("\n");
    }
    public static void main(String[] args) {

        Node head = null;
        head = insertFirst(head,10);
        head = insertFirst(head,20);
        head = insertFirst(head,30);
        head = insertFirst(head,40);
        head = insertFirst(head,50);

        displayList(head);
    }
}
```



9.2.5. Insert the Node at the end of LinkedList

Lab5.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);
        if(headNode==null){
            return temp;
        }

        //To Reach the Last Node of LL
        Node currentNode = headNode;
        while(currentNode.next != null) {
            currentNode=currentNode.next;
        }

        currentNode.next = temp;

        return headNode;
    }

    static void displayList(Node headNode) {
        Node currentNode = headNode;
        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    displayList(head);  
}
```

9.2.6. Insert the Node at given position of LinkedList

Lab6.java

```
package com.jlcindia.linkedlist;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab6 {  
    static Node insert(Node headNode,int position, int data) {  
  
        Node temp = new Node(data);  
  
        if(position==1){  
            temp.next = headNode;  
            return temp;  
        }  
  
        Node currentNode = headNode;  
        for(int i=1;i<=position-2 && currentNode != null;i++) {  
            currentNode=currentNode.next;  
        }  
    }  
}
```



```
        if(currentNode==null) {
            return headNode;
        }

        temp.next=currentNode.next;
        currentNode.next = temp;

        return headNode;
    }
    static void displayList(Node headNode) {
        //Copy from Lab5
    }
    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }
    public static void main(String[] args) {
        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);

        displayList(head);
        System.out.println("-----");

        head = insert(head,3,99);
        displayList(head);
        System.out.println("-----");
        head = insert(head,4,88);
        displayList(head);
    }
}
```



9.2.7. Delete the First Node of LinkedList

Lab7.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab7 {

    static Node deleteFirst(Node headNode) {

        if(headNode==null)
            return null;

        Node newHead = headNode.next;
        headNode.next = null;
        return newHead;
    }

    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }

    static void displayList(Node headNode) {
        //Copy from Lab5
    }

    public static void main(String[] args) {

        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);
    }
}
```



```
        displayList(head);
        System.out.println("-----");

        head= deleteFirst(head);
        displayList(head);
        System.out.println("-----");
        head= deleteFirst(head);
        displayList(head);
    }
}
```



9.2.8. Delete the Last Node of LinkedList

Lab8.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {

    static Node deleteLast(Node headNode) {

        if (headNode == null)
            return null;

        if(headNode.next==null) {
            return null;
        }

        Node currentNode = headNode;
        while (currentNode.next.next != null) {
            currentNode = currentNode.next;
        }

        currentNode.next = null;
        return headNode;
    }
    static Node insertLast(Node headNode, int data) {

        //Copy from Previous Labs
    }

    static void displayList(Node headNode) {

        //Copy from Previous Labs
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    head = deleteLast(head);  
    displayList(head);  
    System.out.println("-----");  
    head = deleteLast(head);  
    displayList(head);  
}  
}
```



9.2.9. Delete the Node at given position of S.L.L

Lab9.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab9 {
    static Node delete(Node headNode, int postion) {

        if (headNode == null)
            return null;

        if (headNode.next == null)
            return null;

        if (postion == 1) {
            return headNode.next;
        }

        Node currentCode = headNode;

        for (int i = 1; i <= postion - 2 && currentCode!=null ; i++) {
            currentCode = currentCode.next;
        }

        if(currentCode==null) {
            return headNode;
        }

        currentCode.next=currentCode.next.next;

        return headNode;
    }
}
```



```
static Node insertLast(Node headNode, int data) {  
  
    //Copy from Previous Labs  
}  
  
static void displayList(Node headNode) {  
  
    //Copy from Previous Labs  
}  
public static void main(String[] args) {  
  
    Node head = null;  
  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    head = delete(head, 2);  
    displayList(head);  
    System.out.println("-----");  
    head = delete(head, 3);  
    displayList(head);  
  
    }  
}
```



9.2.10. Search the Node in LinkedList

Lab10.java

```
package com.jlccindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab10 {

    static int search(Node headNode, int element) {

        int position=1;

        Node currentNode = headNode;
        while(currentNode!=null) {
            if(currentNode.data==element) {
                return position;
            }else {
                currentNode=currentNode.next;
                position++;
            }
        }

        return -1;
    }

    static Node insertLast(Node headNode, int data) {

        //Copy from Previous Labs
    }

    static void displayList(Node headNode) {

        //Copy from Previous Labs
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    int x = search(head, 20);  
    System.out.println(x);  
    System.out.println("-----");  
    x = search(head, 40);  
    System.out.println(x);  
    System.out.println("-----");  
    x = search(head, 70);  
    System.out.println(x);  
  
    }  
}
```



9.2.11. Design MySinglyLinkedList

Lab11.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
public class MySinglyLinkedList {

    Node headNode = null;
    int nodeCount = 0;

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = this.headNode;
        while (currentNode != null) {
            str = str + "" + currentNode.data + ", ";
            currentNode = currentNode.next;
        }
        str = str.substring(0, str.length() - 2);
        str = str + "]";
        return str;
    }
}
```



```
public boolean isEmpty() {  
  
    return nodeCount == 0;  
}  
  
public void clear() {  
  
    Node currentNode=headNode;  
    while(currentNode!=null) {  
        Node temp = currentNode.next;  
  
        currentNode.next=null;  
        currentNode=temp;  
    }  
  
    this.headNode = null;  
    this.nodeCount = 0;  
}  
  
public int size() {  
  
    return nodeCount;  
}  
  
public void insertFirst(int data) {  
  
    Node temp = new Node(data);  
    temp.next = this.headNode;  
    this.headNode = temp;  
    nodeCount++;  
  
}
```



```
public void insertLast(int data) {
```

```
    Node temp = new Node(data);
    if (this.headNode == null) {
        this.headNode = temp;
    }

    // To Reach the Last Node of LL
    Node currentNode = headNode;
    while (currentNode.next != null) {
        currentNode = currentNode.next;
    }

    currentNode.next = temp;
    nodeCount++;
}
```

```
public void insert(int position, int data) {
```

```
    Node temp = new Node(data);

    if (headNode == null) {
        this.headNode = temp;
        return;
    }

    if (position == 1) {
        temp.next = headNode;
        this.headNode = temp;
        return;
    }

    Node currentNode = headNode;
    for (int i = 1; i <= position - 2 && currentNode != null; i++) {
        currentNode = currentNode.next;
    }
}
```



```
        if (currentNode == null) {
            return;
        }

        temp.next = currentNode.next;
        currentNode.next = temp;

        nodeCount++;
    }

    public void deleteFirst() {

        if (this.headNode == null) {
            return;
        }

        this.headNode = headNode.next;
        nodeCount--;
    }

    public void deleteLast() {

        // When No Nodes
        if (this.headNode == null)
            return;

        // When One Node is there
        if (headNode.next == null) {
            this.headNode = null;
            nodeCount--;
            return;
        }

        // When two or more Nodes are there
        Node currentNode = headNode;
        while (currentNode.next.next != null) {
```



```
        currentNode = currentNode.next;
    }

    currentNode.next = null;
    nodeCount--;

}

public void delete(int postion) {

    // 1.Empty List
    if (headNode == null) {
        this.headNode = null;
        return;
    }

    // 2.List with One Node
    if (headNode.next == null) {
        this.headNode = null;
        nodeCount--;
        return;
    }

    // 3. List with 2 or more Nodes
    // Deleting 1st Node
    if (postion == 1) {
        this.headNode = headNode.next;
        nodeCount--;
        return;
    }

    // 4. List with 2 or more Nodes
    Node currentCode = headNode;

    for (int i = 1; i <= postion - 2 && currentCode != null; i++) {
        currentCode = currentCode.next;
    }
}
```



```
// 5. Position is not Present
if (currentCode == null) {
    return;
}

currentCode.next = currentCode.next.next;
nodeCount--;
}

public int search(int element) {

    int position = 1;

    Node currentNode = this.headNode;
    while (currentNode != null) {
        if (currentNode.data == element) {
            return position;
        } else {
            currentNode = currentNode.next;
            position++;
        }
    }

    return -1;

}
}
```



```
public class Lab11 {  
  
    public static void main(String[] args) {  
  
        MySinglyLinkedList mylist=new MySinglyLinkedList();  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.insertFirst(11);  
        mylist.insertFirst(22);  
        mylist.insertFirst(33);  
  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.insertLast(40);  
        mylist.insertLast(50);  
  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.insert(4, 99);  
        mylist.insert(5, 88);  
  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.deleteFirst();  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
    }  
}
```



```
mylist.deleteLast();
System.out.println("-----");
System.out.println(mylist);
System.out.println(mylist.size());

mylist.delete(2);
System.out.println("-----");
System.out.println(mylist);
System.out.println(mylist.size());
System.out.println("-----");

System.out.println(mylist.search(99));
System.out.println(mylist.search(77));

System.out.println("-----");
System.out.println(mylist.isEmpty());
System.out.println(mylist.size());

mylist.clear();

System.out.println(mylist.isEmpty());
System.out.println(mylist.size());

}

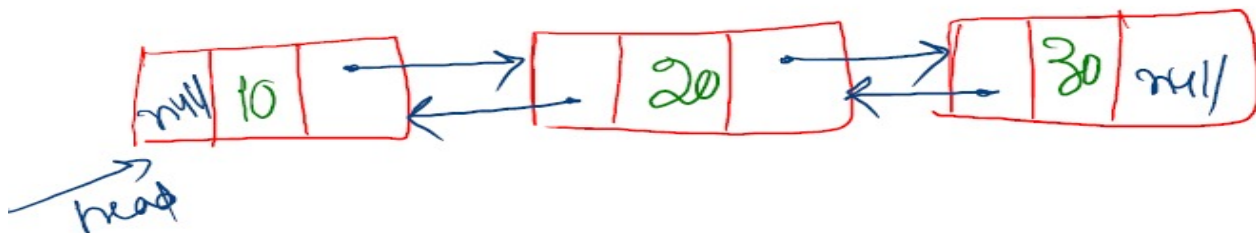
}
```



9.3. Doubly Linked List

- ◆ Doubly Linked List consists of a number of nodes
- ◆ Each Node has will have 3 parts
 - a) Address of Previous Node
 - b) Data
 - c) Address of Next Node
- ◆ Next Address of the Last node in the list is NULL, which indicates that No Next Element.
- ◆ Previous Address of the First/Head node in the list is NULL, which indicates that No Previous element.

Ex:



- ◆ Following is a type declaration for a linked list:

```
public class Node {  
  
    int data;  
    Node next;  
    Node prev;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
        this.prev=null;  
    }  
}
```

- ◆ Main Doubly Linked Lists Operations
 - ✓ Traverse: Access the elements in the List
 - ✓ Count: returns the number of elements in the List
 - ✓ Find nth node from the end of the List
 - ✓ Insert: inserts an element into the List
 - ✓ Delete: removes the specified position element from the List
 - ✓ Reverse the List

9.3.1. Traverse the Doubly LinkedList in forward order

Lab12.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {

    int data;
    Node next;
    Node prev;

    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev=null;
    }

}
```



```
public class Lab12 {

    static String toString(Node headNode) {
        if (headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = headNode;
        while (currentNode != null) {
            str = str + "" + currentNode.data + " , ";
            currentNode = currentNode.next;
        }
        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);

        head.next = node2;
        node2.prev = head;

        node2.next = node3;
        node3.prev = node2;

        System.out.println(toString(head));
    }
}
```



9.3.2. Traverse the Doubly LinkedList in Reverse order

Lab13.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab13 {

    static String toString(Node headNode) {

        if (headNode == null) {
            return "[]";
        }

        Node tailNode = headNode;

        while(tailNode.next!=null) {
            tailNode=tailNode.next;
        }

        String str = "[";
        Node currentNode = tailNode;
        while (currentNode != null) {
            str = str + "" + currentNode.data + " , ";
            currentNode = currentNode.prev;
        }

        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = new Node(10);  
    Node node2 = new Node(20);  
    Node node3 = new Node(30);  
  
    head.next = node2;  
    node2.prev = head;  
  
    node2.next = node3;  
    node3.prev = node2;  
  
    System.out.println(toString(head));  
  
}  
}
```



9.3.3. Insert the Node at beginning of D.L.L

Lab14.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab14 {

    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp; //head node changed
            return headNode;
        }

        //2.List with 1 or more Nodes
        temp.next=headNode;
        headNode.prev=temp;

        headNode=temp; //head node changed

        return headNode;
    }
}
```



```
static String toString(Node headNode) {

    if (headNode == null) {
        return "[]";
    }

    String str = "[";
    Node currentNode = headNode;
    while (currentNode != null) {
        str = str + "" + currentNode.data + ", ";
        currentNode = currentNode.next;
    }
    str = str.substring(0, str.length() - 2);
    str = str + "]";

    return str;
}

public static void main(String[] args) {

    Node head=null;
    head = insertFirst(head,10);
    head = insertFirst(head,20);
    head = insertFirst(head,30);

    System.out.println(toString(head));

}

}
```



9.3.4. Insert the Node at end of D.L.L

Lab15.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab15 {

    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp; //head node changed
            return headNode;
        }

        //2.List with 1 or more Nodes
        Node currentNode = headNode;
        while (currentNode.next != null) {
            currentNode = currentNode.next;
        }

        currentNode.next=temp;
        temp.prev=currentNode;
        return headNode;
    }
}
```



```
static String toString(Node headNode) {

    if (headNode == null) {
        return "[]";
    }

    String str = "[";
    Node currentNode = headNode;
    while (currentNode != null) {
        str = str + "" + currentNode.data + ", ";
        currentNode = currentNode.next;
    }
    str = str.substring(0, str.length() - 2);
    str = str + "]";

    return str;
}

public static void main(String[] args) {

    Node head=null;
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,99);

    System.out.println(toString(head));

}

}
```



9.3.5. Insert the Node at given position in D.L.L

Lab16.java

```
package com.jlccindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab16 {

    static Node insert(Node headNode, int position, int data) {

        //1. Create New Node
        Node temp = new Node(data);

        //2.when head is null
        if(headNode==null) {
            return temp;
        }

        //3. If position is 1
        if(position==1){
            temp.next = headNode;
            headNode.prev=temp;
            return temp;
        }

        //4. Move the position -1 Node
        Node currentNode = headNode;
        for(int i=1;i<=position-2 && currentNode != null;i++) {
            currentNode=currentNode.next;
        }

        //5. If Position is out of range
        if(currentNode==null) {
            return headNode;
        }
    }
}
```



```
//6. Insert at given Position
temp.next=currentNode.next;
temp.prev=currentNode;

currentNode.next.prev=temp;
currentNode.next = temp;

return headNode;

}

static Node insertLast(Node headNode,int data) {
    // Copy from Lab15
}

static String toString(Node headNode) {
    // Copy from Lab15
}

public static void main(String[] args) {

    Node head=null;
    head = insert(head,3,55);
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,40);
    head = insertLast(head,50);

    System.out.println(toString(head));
    head = insert(head,3,99); //
    System.out.println(toString(head));
}
}
```



9.3.6. Delete the First Node in D.L.L

Lab17.java

```
package com.jlccindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab17 {

    static Node deleteFirst(Node headNode) {

        //1. If head is null
        if(headNode==null) {
            return null;
        }

        //2. If One Node is there
        if(headNode.next==null){
            return null;
        }

        //3.If Head is Not Null
        Node temp = headNode;

        headNode = headNode.next;
        headNode.prev=null;

        temp.next = null;

        return headNode;
    }

    static Node insertLast(Node headNode,int data) {
        // Copy from Lab15
    }
}
```



```
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head=null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    System.out.println(toString(head));  
    head = deleteFirst(head); //10  
    System.out.println(toString(head));  
    head = deleteFirst(head); //20  
    System.out.println(toString(head));  
  
    }  
}
```



9.3.7. Delete the Last Node in D.L.L

Lab18.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab18 {

    static Node deleteLast(Node headNode) {

        //1. If head is null
        if(headNode==null) {
            return null;
        }

        //2. If One Node is there
        if(headNode.next==null){
            return null;
        }

        //3. Move to Last but one
        Node currentNode = headNode;
        while (currentNode.next.next != null) {
            currentNode = currentNode.next;
        }

        Node temp = currentNode.next;
        temp.prev=null;

        currentNode.next = null;
        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15
```

```
}
```

```
static String toString(Node headNode) {  
    // Copy from Lab15
```

```
}
```

```
public static void main(String[] args) {
```

```
    Node head=null;
```

```
    head = insertLast(head,10);
```

```
    head = insertLast(head,20);
```

```
    head = insertLast(head,30);
```

```
    head = insertLast(head,40);
```

```
    head = insertLast(head,50);
```

```
    System.out.println(toString(head));
```

```
    head = deleteLast(head); //50
```

```
    System.out.println(toString(head));
```

```
    head = deleteLast(head); //40
```

```
    System.out.println(toString(head));
```

```
}
```

```
}
```



9.3.8. Delete the Node at given position in D.L.L

Lab19.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab19 {

    static Node delete(Node headNode, int position) {

        // 1.head is null
        if (headNode == null)
            return null;

        // 2.One Node
        if (headNode.next == null)
            return null;

        // 3.delete first node
        if (position == 1) {
            Node temp = headNode;

            headNode = headNode.next;
            headNode.prev = null;

            temp.next = null;
            return headNode;
        }
    }
}
```



```
// 4. Move to Position -1
Node currentCode = headNode;

for (int i = 1; i <= position - 2 && currentCode != null; i++) {
    currentCode = currentCode.next;
}

// 5.Position is out of Range
if (currentCode == null || currentCode.next==null) {
    return headNode;
}

Node temp = currentCode.next;

currentCode.next = currentCode.next.next;
if (currentCode.next != null) {
    currentCode.next.prev = currentCode;
}

temp.next = null;
temp.prev = null;

return headNode;
}

static Node insertLast(Node headNode,int data) {
    // Copy from Lab15
}

static String toString(Node headNode) {
    // Copy from Lab15
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    head = delete(head, 3); // 30  
    System.out.println(toString(head));  
    head = delete(head, 4); // 50  
    System.out.println(toString(head));  
    head = delete(head, 4); // No Deletion  
    System.out.println(toString(head));  
    head = delete(head, 3); // 40  
    System.out.println(toString(head));  
}  
}
```



9.3.9. Search the Node in D.L.L from beginnig

Lab20.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab20{

    static int search(Node headNode, int element) {

        int position =1;

        Node currentNode = headNode;

        while(currentNode!=null) {
            if( currentNode.data == element) {
                return position;
            }else {
                position++;
                currentNode = currentNode.next;
            }
        }
        return -1;
    }

    static Node insertLast(Node headNode,int data) {
        // Copy from Lab15
    }

    static String toString(Node headNode) {
        // Copy from Lab15
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    int x = search(head, 30); //3  
    System.out.println(x);  
    x = search(head, 50); //5  
    System.out.println(x);  
    x = search(head, 60); //-1  
    System.out.println(x);  
}  
}
```



9.3.10. Search the Node in D.L.L from ending

Lab21.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab21{

    static int search(Node headNode, int element) {

        Node currentNode = headNode;
        int nodeCount = 1;

        //Move to Last Node
        while(currentNode.next != null) {
            currentNode = currentNode.next;
            nodeCount++;
        }

        while(currentNode!=null) {
            if( currentNode.data == element) {
                return nodeCount;
            }else {
                nodeCount--;
                currentNode = currentNode.prev;
            }
        }

        return -1;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}
```

```
static String toString(Node headNode) {  
    // Copy from Lab15  
}
```

```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 30);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    int x = search(head, 30); //3  
    System.out.println(x);  
    x = search(head, 50); //5  
    System.out.println(x);  
    x = search(head, 70); //-1  
    System.out.println(x);  
}
```

```
}
```



9.3.11. Reverse the D.L.L

Lab22.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab22 {
    static Node reverse(Node headNode) {

        // 1. Empty List
        if (headNode == null) {
            return null;
        }
        // 2. One Node List
        if (headNode.next == null) {
            return headNode;
        }

        Node tempNode = null;

        Node currentNode = headNode;
        while(currentNode!=null) {
            //Swap
            tempNode = currentNode.prev;
            currentNode.prev= currentNode.next;
            currentNode.next = tempNode;

            currentNode = currentNode.prev;
        }

        headNode = tempNode.prev;
        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}  
  
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    head = reverse(head);  
    System.out.println(toString(head));  
  
}  
}
```



9.3.12. Design MyDoublyLinkedList

Lab23.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {
    int data;
    Node next;
    Node prev;

    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev=null;
    }
}

public class MyDoublyLinkedList {

    Node headNode = null;
    int nodeCount = 0;

    public MyDoublyLinkedList() {
    }

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = this.headNode;
```



```
        while (currentNode != null) {
            str = str + "" + currentNode.data + ", ";
            currentNode = currentNode.next;
        }
        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }

    public boolean isEmpty() {
        return this.nodeCount == 0;
    }

    public int size() {
        return this.nodeCount;
    }

    public void clear() {

        Node currentNode = this.headNode;
        while (currentNode != null) {
            Node temp = currentNode.next;

            currentNode.next = null;
            currentNode.prev = null;

            currentNode = temp;
        }

        this.headNode = null;
        this.nodeCount = 0;
    }
}
```



```
public void insertFirst(int data) {
```

```
    Node temp = new Node(data);
```

```
    // 1.Empty List
```

```
    if (this.headNode == null) {
```

```
        this.headNode = temp;
```

```
        nodeCount++;
```

```
        return;
```

```
    }
```

```
    // 2.List with 1 or more Nodes
```

```
    temp.next = this.headNode;
```

```
    this.headNode.prev = temp;
```

```
    this.headNode = temp;
```

```
    nodeCount++;
```

```
}
```

```
public void insertLast(int data) {
```

```
    Node temp = new Node(data);
```

```
    // 1.Empty List
```

```
    if (this.headNode == null) {
```

```
        this.headNode = temp;
```

```
        nodeCount++;
```

```
        return;
```

```
    }
```

```
    // 2.List with 1 or more Nodes
```

```
    Node currentNode = this.headNode;
```

```
    while (currentNode.next != null) {
```

```
        currentNode = currentNode.next;
```

```
    }
```



```
        currentNode.next = temp;
        temp.prev = currentNode;
        nodeCount++;
    }

    public void insert(int position, int data) {

        // 1. Create New Node
        Node temp = new Node(data);

        // 2.Empty List
        if (this.headNode == null) {
            this.headNode = temp;
            nodeCount++;
            return;
        }

        // 3. If position is 1
        if (position == 1) {
            temp.next = this.headNode;
            this.headNode.prev = temp;
            this.headNode = temp;
            nodeCount++;
            return;
        }

        // 4. Move the position -1 Node
        Node currentNode = this.headNode;
        for (int i = 1; i <= position - 2 && currentNode != null; i++) {
            currentNode = currentNode.next;
        }

        // 5. If Position is out of range
        if (currentNode == null) {
            return;
        }
    }
}
```



```
// 6. Insert at given Position
temp.next = currentNode.next;
temp.prev = currentNode;

currentNode.next.prev = temp;
currentNode.next = temp;

nodeCount++;

}

public void deleteFirst() {

    // 1. If head is null
    if (this.headNode == null) {
        this.headNode = null;
        return;
    }

    // 2. If One Node is there
    if (this.headNode.next == null) {
        this.headNode = null;
        nodeCount--;
        return;
    }

    // 3. when more nodes are there
    Node temp = this.headNode;

    this.headNode = this.headNode.next;
    headNode.prev = null;

    temp.next = null;
    nodeCount--;

}
```



```
public void deleteLast() {
```

```
    // 1. If head is null
    if (this.headNode == null) {
        return;
    }

    // 2. If One Node is there
    if (this.headNode.next == null) {
        this.headNode = null;
        nodeCount--;
        return;
    }

    // 3. When two or More Nodes
    Node currentNode = this.headNode;
    while (currentNode.next.next != null) {
        currentNode = currentNode.next;
    }

    Node temp = currentNode.next;
    temp.prev = null;

    currentNode.next = null;
    nodeCount--;

}
```

```
public void delete(int position) {
```

```
    // 1.head is null
    if (this.headNode == null) {
        this.headNode = null;
        return;
    }
}
```



```
// 2.One Node
if (this.headNode.next == null) {
    this.headNode = null;
    nodeCount--;
    return;
}

// 3.delete first node
if (position == 1) {
    Node temp = this.headNode;

    this.headNode = this.headNode.next;
    this.headNode.prev = null;

    temp.next = null;
    nodeCount--;

    return;
}

// 4. Move to Position -1
Node currentCode = this.headNode;

for (int i = 1; i <= position - 2 && currentCode != null; i++) {
    currentCode = currentCode.next;
}

// 5.Position is out of Range
if (currentCode == null || currentCode.next == null) {
    return;
}

Node temp = currentCode.next;

currentCode.next = currentCode.next.next;
if (currentCode.next != null) {
    currentCode.next.prev = currentCode;
}
```



```
    }

    temp.next = null;
    temp.prev = null;

    nodeCount--;

}

public int searchFirst(int element) {

    int position = 1;
    Node currentNode = headNode;

    while (currentNode != null) {
        if (currentNode.data == element) {
            return position;
        } else {
            position++;
            currentNode = currentNode.next;
        }
    }
    return -1;
}

public int searchLast(int element) {

    Node currentNode = headNode;
    int nodeCount = 1;

    // Move to Last Node
    while (currentNode.next != null) {
        currentNode = currentNode.next;
        nodeCount++;
    }

}
```



```
        while (currentNode != null) {
            if (currentNode.data == element) {
                return nodeCount;
            } else {
                nodeCount--;
                currentNode = currentNode.prev;
            }
        }
        return -1;
    }

    public void reverse() {
        // 1. Empty List
        if (this.headNode == null) {
            return;
        }
        // 2. One Node List
        if (this.headNode.next == null) {
            return;
        }

        Node tempNode = null;

        Node currentNode = this.headNode;
        while (currentNode != null) {
            // Swap
            tempNode = currentNode.prev;
            currentNode.prev = currentNode.next;
            currentNode.next = tempNode;

            currentNode = currentNode.prev;
        }
        this.headNode = tempNode.prev;
    }
}
```



```
public class Lab23 {  
  
    public static void main(String[] args) {  
  
        MyDoublyLinkedList mylist = new MyDoublyLinkedList();  
  
        System.out.println("-----1-----");  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insertFirst(10);  
        mylist.insertFirst(20);  
        mylist.insertFirst(30);  
  
        System.out.println("-----2-----");  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insertLast(88);  
        mylist.insertLast(99);  
  
        System.out.println("-----3-----");  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insert(4, 77);  
  
        System.out.println("-----4-----");  
        System.out.println(mylist);  
    }  
}
```



```
mylist.deleteFirst();

System.out.println("-----5-----");
System.out.println(mylist);

mylist.deleteLast();

System.out.println("-----6-----");
System.out.println(mylist);

mylist.delete(2);

System.out.println("-----7-----");
System.out.println(mylist);

mylist.insertLast(77);
mylist.insertLast(99);

System.out.println("-----8-----");
System.out.println(mylist);

System.out.println("-----9-----");
System.out.println(mylist.searchFirst(77));
System.out.println(mylist.searchLast(77));
System.out.println(mylist.searchFirst(55));
System.out.println(mylist.searchLast(55));

mylist.reverse();
System.out.println("-----10-----");
System.out.println(mylist);
}
```

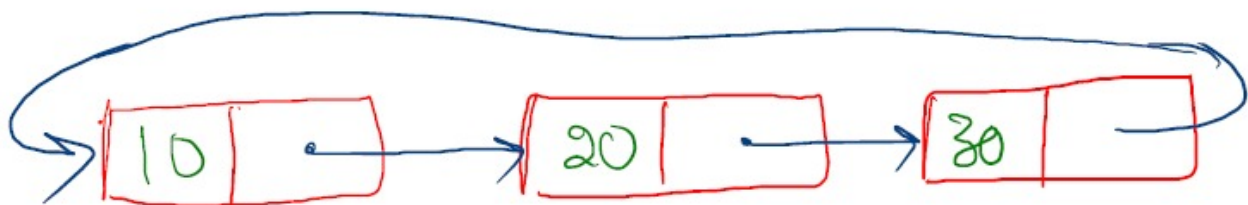
9.4. Circular Linked List

- ♦ In Singly Linked Lists and Doubly Linked Lists, the end of lists are indicated with NULL value.
- ♦ But Circular Linked Lists do not have ends.
- ♦ In Circular Linked Lists, each node has a successor.
- ♦ There is no node with NULL pointer in a Circular Linked Lists.
- ♦ Circular Linked Lists are useful in some cases like Round Robin Algorithm
- ♦ Circular Linked Lists can be implemented in two ways
 - a) Circular Singly Linked Lists
 - b) Circular Doubly Linked Lists

9.5. Circular Singly Linked List

- ♦ Circular Singly Linked List are same as Singly Linked List without having node with NULL.
- ♦ Circular Singly Linked List consists of a number of nodes without having node with NULL.
- ♦ Each Node has will have 2 parts
 - a) Data
 - b) Address of Next Node
- ♦ Address of the Last node in the list is First Node Address

Ex:



- ◆ Following is a type declaration for the Node:

```
public class Node {  
  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

- ◆ Main Circular Singly Linked List Operations
 - ✓ Traverse: Access the elements in the List
 - ✓ Count: returns the number of elements in the List
 - ✓ Find nth node from the end of the List
 - ✓ Insert: inserts an element into the List
 - ✓ Delete: removes the specified position element from the List

9.5.1. Traverse the Circular Singly LinkedList

Lab24A.java

```
package com.jlccindia.circular.singly.linkedlist;  
  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
  
// Traverse the Circular Singly LinkedList  
  
public class Lab24A {  
  
    static void displayList(Node headNode) {
```



```
        if(headNode==null) {
            return ;
        }

        System.out.print(headNode.data+"\t");

        Node currentNode = headNode.next;
        while(currentNode != headNode) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);
        Node node4 = new Node(40);

        head.next=node2;
        node2.next=node3;
        node3.next=node4;
        node4.next=head;

        displayList(head);
    }
}
```



9.5.2. Traverse the Circular Singly LinkedList

Lab24B.java

```
package com.jlccindia.circular.singly.linkedlist;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Traverse the Circular Singly LinkedList
//Prefer this

public class Lab24B {

    static void displayList(Node headNode) {
        if (headNode == null) {
            return;
        }

        Node currentNode = headNode;

        do {
            System.out.print(currentNode.data + "\t");
            currentNode = currentNode.next;
        } while (currentNode != headNode);

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);
        Node node4 = new Node(40);
```



```
        head.next = node2;
        node2.next = node3;
        node3.next = node4;
        node4.next = head;

        displayList(head);
    }
}
```

9.5.3. Count the Nodes in Circular Singly LinkedList

Lab25.java

```
package com.jlcindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Count the Nodes in Circular Singly LinkedList
public class Lab25{

    static int length(Node headNode) {

        int length=0;
        Node currentNode = headNode;

        do {
            length++;
            currentNode = currentNode.next;
        } while (currentNode != headNode);

        return length;
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = new Node(10);  
    Node node2 = new Node(20);  
    Node node3 = new Node(30);  
    Node node4 = new Node(40);  
    Node node5=new Node(50);  
  
    head.next = node2;  
    node2.next = node3;  
    node3.next = node4;  
    node4.next = node5;  
    node5.next=head;  
  
    int len=length(head);  
    System.out.println(len);  
  
}  
}
```



9.5.4. Insert the Node at beginning of Circular Singly LinkedList

Lab26A.java

```
package com.jlcindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Insert the Node at beginning of Circular Singly LinkedList
public class Lab26A {

    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp;
            headNode.next=headNode;
            return headNode;
        }

        Node currentNode = headNode;
        while(currentNode.next!=headNode) {
            currentNode=currentNode.next;
        }

        temp.next=headNode;
        currentNode.next = temp;

        //Head Changed
        headNode=temp;

        return headNode;
    }
}
```



```
static void displayList(Node headNode) {

    Node currentNode = headNode;

    do {
        System.out.print(currentNode.data + "\t");
        currentNode = currentNode.next;
    } while (currentNode != headNode);

    System.out.println("\n");
}

public static void main(String[] args) {

    //1.Empty List
    Node head= null;
    head = insertFirst(head,10);
    displayList(head);

    System.out.println("-----");
    //2. One Node List
    head = insertFirst(head,20);
    displayList(head);

    System.out.println("-----");
    //3. More Nodes
    head = insertFirst(head,30);
    head = insertFirst(head,40);
    head = insertFirst(head,50);
    displayList(head);
}

// Time - O(n)
// Space - O(1)
```



9.5.5. Insert the Node at beginning of Circular Singly LinkedList

Lab26B.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Insert the Node at beginning of Circular Singly LinkedList
public class Lab26B {

    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp;
            headNode.next=headNode;
            return headNode;
        }

        temp.next=headNode.next;
        headNode.next=temp;

        //Swap data of 1st and 2nd Nodes
        int x= headNode.data;
        headNode.data= temp.data;
        temp.data=x;

        //Head Not Changed

        return headNode;
    }
}
```



```
static void displayList(Node headNode) {

    Node currentNode = headNode;

    do {
        System.out.print(currentNode.data + "\t");
        currentNode = currentNode.next;
    } while (currentNode != headNode);

    System.out.println("\n");
}

public static void main(String[] args) {

    //1.Empty List
    Node head= null;
    head = insertFirst(head,10);
    displayList(head);

    System.out.println("-----");
    //2. One Node List
    head = insertFirst(head,20);
    displayList(head);

    System.out.println("-----");
    //3. More Nodes
    head = insertFirst(head,30);
    head = insertFirst(head,40);
    head = insertFirst(head,50);
    displayList(head);
}

// Time - O(1)
// Space - O(1)
```

9.5.6. Insert the Node at ending of Circular Singly LinkedList

Lab27A.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Insert the Node at ending of Circular Singly LinkedList
public class Lab27A {

    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp;
            headNode.next=headNode;
            return headNode;
        }

        Node currentNode = headNode;
        while(currentNode.next!=headNode) {
            currentNode=currentNode.next;
        }

        temp.next=headNode;
        currentNode.next=temp;

        //Head Not Changed

        return headNode;
    }
}
```



```
static void displayList(Node headNode) {

    Node currentNode = headNode;

    do {
        System.out.print(currentNode.data + "\t");
        currentNode = currentNode.next;
    } while (currentNode != headNode);

    System.out.println("\n");
}

public static void main(String[] args) {

    //1.Empty List
    Node head= null;
    head = insertLast(head,10);
    displayList(head);

    System.out.println("-----");
    //2. One Node List
    head = insertLast(head,20);
    displayList(head);

    System.out.println("-----");
    //3. More Nodes
    head = insertLast(head,30);
    head = insertLast(head,40);
    head = insertLast(head,99);
    displayList(head);
}

// Time - O(n)
// Space - O(1)
```



9.5.7 Insert the Node at ending of Circular Singly LinkedList

Lab27B.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Insert the Node at ending of Circular Singly LinkedList
public class Lab27B {

    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp;
            headNode.next=headNode;
            return headNode;
        }

        temp.next=headNode.next;
        headNode.next=temp;

        //Swap data of 1st and 2nd Nodes
        int x= headNode.data;
        headNode.data= temp.data;
        temp.data=x;

        //Head has to be Changed
        headNode = temp;

        return headNode;
    }
}
```



```
static void displayList(Node headNode) {

    Node currentNode = headNode;

    do {
        System.out.print(currentNode.data + "\t");
        currentNode = currentNode.next;
    } while (currentNode != headNode);

    System.out.println("\n");
}

public static void main(String[] args) {

    //1.Empty List
    Node head= null;
    head = insertLast(head,10);
    displayList(head);

    System.out.println("-----");
    //2. One Node List
    head = insertLast(head,20);
    displayList(head);

    System.out.println("-----");
    //3. More Nodes
    head = insertLast(head,30);
    head = insertLast(head,40);
    head = insertLast(head,99);
    displayList(head);
}

// Time - O(1)
// Space - O(1)
```



9.5.8 Insert the Node at given position of Circular Singly LinkedList

Lab28.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Insert the Node at given position of Circular Singly LinkedList
public class Lab28 {
    static Node insert(Node headNode,int position, int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            return headNode;
        }

        //2. Position = 1
        if(position==1){
            return insertFirst(headNode, data);
        }

        Node currentNode = headNode;
        for(int i=1;i<=position-2 && currentNode.next != headNode;i++) {
            currentNode=currentNode.next;
        }

        if(currentNode.next==headNode) {
            return headNode;
        }

        temp.next=currentNode.next;
        currentNode.next = temp;

        //HeadNode is Not Changed
    }
}
```



```
        return headNode;
    }

    static Node insertFirst(Node headNode,int data) {
        Refer Lab26B
    }

    static Node insertLast(Node headNode,int data) {
        Refer Lab27B
    }

    static void displayList(Node headNode) {
        Refer Lab27B
    }

    public static void main(String[] args) {

        Node head = null;
        head = insertFirst(head,50);
        head = insertFirst(head,40);
        head = insertFirst(head,30);
        head = insertFirst(head,20);
        head = insertFirst(head,10);

        displayList(head);
        System.out.println("-----");

        head = insert(head,1,99);
        displayList(head);
    }
}
// Time - O(n)
//Space - O(1)
```



9.5.9 Delete the First Node in Circular Singly LinkedList

Lab29A.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Delete the First Node in Circular Singly LinkedList
public class Lab29A {
    static Node deleteFirst(Node headNode) {

        //1. Empty List
        if(headNode==null) {
            return null;
        }

        //2. One Node List
        if(headNode.next==headNode) {
            headNode.next=null;
            return null;
        }

        //3.List with 2 or more Nodes
        Node currentNode = headNode;
        while(currentNode.next!=headNode) {
            currentNode=currentNode.next;
        }

        currentNode.next = headNode.next;
        headNode.next=null;

        headNode = currentNode.next;

        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {
```

Refer Lab28

```
}
```

```
static void displayList(Node headNode) {
```

Refer Lab28

```
}
```

```
public static void main(String[] args) {
```

```
    Node head=null;
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,40);
    head = insertLast(head,50);
    displayList(head);
    System.out.println("-----");
    head = deleteFirst(head);
    displayList(head);
    System.out.println("-----");
    head = deleteFirst(head);
    displayList(head);
}
```

```
}
```

// Time - O(n)

// Space - O(1)

9.5.10 Delete the First Node in Circular Singly LinkedList

Lab29B.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Delete the First Node in Circular Singly LinkedList
public class Lab29B {

    static Node deleteFirst(Node headNode) {

        //1. Empty List
        if(headNode==null) {
            return null;
        }

        //2. One Node List
        if(headNode.next==headNode) {
            headNode.next=null;
            return null;
        }

        //3.List with 2 or more Nodes

        headNode.data=headNode.next.data;
        Node temp = headNode.next;

        headNode.next=headNode.next.next;
        temp.next=null;

        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {
```

Refer Lab28

```
}
```

```
static void displayList(Node headNode) {
```

Refer Lab28

```
}
```

```
public static void main(String[] args) {
```

```
    Node head=null;
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,40);
    head = insertLast(head,50);
    displayList(head);
    System.out.println("-----");
    head = deleteFirst(head);
    displayList(head);
    System.out.println("-----");
    head = deleteFirst(head);
    displayList(head);
```

```
}
```

```
}
```

```
// Time - O(1)
```

```
// Space - O(1)
```



9.5.11. Delete the Last Node in Circular Singly LinkedList

Lab30.java

```
package com.jlccindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Delete the Last Node in Circular Singly LinkedList
public class Lab30{

    static Node deleteLast(Node headNode) {

        //1. Empty List
        if(headNode==null) {
            return null;
        }

        //2. One Node List
        if(headNode.next==headNode) {
            headNode.next=null;
            return null;
        }

        //3.List with 2 or more Nodes
        Node currentNode = headNode;
        while(currentNode.next.next!=headNode) {
            currentNode=currentNode.next;
        }

        currentNode.next.next=null;
        currentNode.next=headNode;

        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    Refer Lab28  
}  
  
static void displayList(Node headNode) {  
    Refer Lab28  
}  
  
public static void main(String[] args) {  
  
    Node head=null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
    displayList(head);  
  
    System.out.println("-----");  
    head = deleteLast(head);  
    displayList(head);  
    System.out.println("-----");  
    head = deleteLast(head);  
    displayList(head);  
}  
}  
  
// Time - O(n)  
// Space - O(1)
```



9.5.12. Delete the Node at given position in Circular Singly LinkedList

Lab31.java

```
package com.jlcindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Delete the Node at given position in Circular Singly LinkedList
public class Lab31{
    static Node delete(Node headNode,int position) {

        //1. Empty List
        if(headNode==null) {
            return headNode;
        }

        //2. One Node List
        if(headNode.next==headNode) {
            return headNode;
        }

        //3.List with 2 or more Nodes
        Node currentNode = headNode;
        for(int i=1;i<=position-2 && currentNode.next!=headNode;i++) {
            currentNode=currentNode.next;
        }

        //When poistion is out of range
        if(currentNode.next==headNode) {
            return headNode;
        }

        //When poistion is in range
        Node temp = currentNode.next.next;
        currentNode.next.next=null;
        currentNode.next=temp;
    }
}
```



```
        return headNode;
    }

    static Node insertLast(Node headNode,int data) {
        Refer Lab28
    }

    static void displayList(Node headNode) {
        Refer Lab28
    }

    public static void main(String[] args) {

        Node head=null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);
        displayList(head);

        System.out.println("-----");
        head = delete(head,9);
        displayList(head);

        System.out.println("-----");
        head = delete(head,4);
        displayList(head);
        System.out.println("-----");
        head = delete(head,2);
        displayList(head);
    }
}
// Time - O(n)
// Space - O(1)
```



9.5.13. Search the Element in Circular Singly LinkedList

Lab32.java

```
package com.jlcindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Search the Element in Circular Singly LinkedList

public class Lab32{

    static int search(Node headNode,int elemenet) {

        if (headNode == null) {
            return -1;
        }

        int position = 1;
        Node currentNode = headNode;

        do {
            if(currentNode.data==elemenet) {
                return position;
            }else {
                position++;
                currentNode=currentNode.next;
            }
        } while (currentNode != headNode);

        return -1;
    }
}
```



```
static Node insertLast(Node headNode,int data) {
```

Refer Lab28

```
}
```

```
static void displayList(Node headNode) {
```

Refer Lab28

```
}
```

```
public static void main(String[] args) {
```

```
    Node head=null;
```

```
    head = insertLast(head,10);
```

```
    head = insertLast(head,20);
```

```
    head = insertLast(head,30);
```

```
    head = insertLast(head,40);
```

```
    head = insertLast(head,50);
```

```
    displayList(head);
```

```
    System.out.println("-----");
```

```
    int x= search(head,99);
```

```
    System.out.println(x);
```

```
    System.out.println("-----");
```

```
    x = search(head,30);
```

```
    System.out.println(x);
```

```
    System.out.println("-----");
```

```
    x = search(head,50);
```

```
    System.out.println(x);
```

```
}
```

```
}
```

```
// Time - O(n)
```

```
// Space - O(1)
```



9.5.14. Design MyCircularSinglyLinkedList

Lab33.java

```
package com.jlcindia.circular.singly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Design MyCircularSinglyLinkedList
public class MyCircularSinglyLinkedList {

    Node headNode = null;
    int nodeCount = 0;

    public MyCircularSinglyLinkedList() { }

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = this.headNode;
        do {
            str = str + "" + currentNode.data + " , ";
            currentNode = currentNode.next;
        } while (currentNode != headNode);

        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }

    public boolean isEmpty() {
        return nodeCount == 0;
    }
}
```



```
public void clear() {
```

```
    Node currentNode = headNode;
    do {
        Node temp = currentNode.next;

        currentNode.next = null;
        currentNode = temp;
    } while (currentNode != headNode);

    this.headNode = null;
    this.nodeCount = 0;
}
```

```
public int size() {
```

```
    return nodeCount;
}
```

```
public void insertFirst(int data) {
```

```
    Node temp = new Node(data);

    // 1.Empty List
    if (headNode == null) {
        headNode = temp;
        headNode.next = headNode;
        nodeCount++;
        return;
    }

    // 2. Non-Empty List
    temp.next = headNode.next;
    headNode.next = temp;

    // Swap data of 1st and 2nd Nodes
    int x = headNode.data;
```



```
        headNode.data = temp.data;
        temp.data = x;

        nodeCount++;

        // Head Not Changed
    }

    public void insertLast(int data) {

        Node temp = new Node(data);

        // 1.Empty List
        if (headNode == null) {
            headNode = temp;
            headNode.next = headNode;
            nodeCount++;
            return;
        }

        // 2.Non-Empty List
        temp.next = headNode.next;
        headNode.next = temp;

        // Swap data of 1st and 2nd Nodes
        int x = headNode.data;
        headNode.data = temp.data;
        temp.data = x;

        // Head has to be Changed
        this.headNode = temp;
        nodeCount++;

    }
```



```
public void insert(int position, int data) {
    Node temp = new Node(data);

    // 1.Empty List
    if (this.headNode == null) {
        this.headNode = temp;
        this.headNode.next = this.headNode;
        nodeCount++;
        return;
    }

    // 2. Position = 1
    if (position == 1) {
        insertFirst(data);
        return;
    }

    // When position is >= 2
    Node currentNode = headNode;
    for (int i = 1; i <= position - 2 && currentNode.next != headNode; i++) {
        currentNode = currentNode.next;
    }

    if (currentNode.next == headNode) {
        return;
    }

    temp.next = currentNode.next;
    currentNode.next = temp;

    // HeadNode is Not Changed
    nodeCount++;
}
```



```
public void deleteFirst() {  
    // 1. Empty List  
    if (headNode == null) {  
        return;  
    }  
  
    // 2. One Node List  
    if (headNode.next == headNode) {  
        headNode.next = null;  
        // Head Changed  
        this.headNode = null;  
        nodeCount--;  
        return;  
    }  
  
    // 3. List with 2 or more Nodes  
  
    headNode.data = headNode.next.data;  
    Node temp = headNode.next;  
  
    headNode.next = headNode.next.next;  
    temp.next = null;  
  
    // Head Not Changed  
    nodeCount--;  
}
```



```
public void deleteLast() {  
  
    // 1. Empty List  
    if (headNode == null) {  
        return;  
    }  
  
    // 2. One Node List  
    if (headNode.next == headNode) {  
        headNode.next = null;  
        // Head Changed  
        this.headNode = null;  
        nodeCount--;  
        return;  
    }  
  
    // 3. List with 2 or more Nodes  
  
    Node currentNode = headNode;  
    while (currentNode.next.next != headNode) {  
        currentNode = currentNode.next;  
    }  
  
    currentNode.next.next = null;  
    currentNode.next = headNode;  
  
    // Head Not Changed  
    nodeCount--;  
}
```



```
public void delete(int position) {
```

```
    // 1. Empty List
```

```
    if (headNode == null) {
```

```
        return;
```

```
    }
```

```
    // 2. One Node List
```

```
    if (headNode.next == headNode) {
```

```
        headNode.next = null;
```

```
        // Head Changed
```

```
        this.headNode = null;
```

```
        nodeCount--;
```

```
        return;
```

```
    }
```

```
    // 3. List with 2 or more Nodes
```

```
    Node currentNode = headNode;
```

```
    for (int i = 1; i <= position - 2 && currentNode.next != headNode; i++) {
```

```
        currentNode = currentNode.next;
```

```
    }
```

```
    // When position is out of range
```

```
    if (currentNode.next == headNode) {
```

```
        return;
```

```
    }
```

```
    // When position is in range
```

```
    Node temp = currentNode.next.next;
```

```
    currentNode.next.next = null;
```

```
    currentNode.next = temp;
```

```
    // Head Changed
```

```
    nodeCount--;
```

```
}
```



```
public int search(int element) {  
  
    if (headNode == null) {  
        return -1;  
    }  
  
    int position = 1;  
    Node currentNode = headNode;  
  
    do {  
        if (currentNode.data == element) {  
            return position;  
        } else {  
            position++;  
            currentNode = currentNode.next;  
        }  
    } while (currentNode != headNode);  
  
    return -1;  
  
}
```

// Test MyCircularSinglyLinkedList

```
public class Lab33{  
    public static void main(String[] args) {  
  
        MyCircularSinglyLinkedList mylist = new MyCircularSinglyLinkedList();  
        System.out.println(mylist.size());  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist);  
  
        mylist.insertFirst(99);  
        mylist.insertFirst(88);  
    }  
}
```



```
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist.isEmpty());  
System.out.println(mylist);
```

```
mylist.clear();
```

```
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist.isEmpty());  
System.out.println(mylist);
```

```
mylist.insertLast(10);  
mylist.insertLast(20);  
mylist.insertLast(30);  
mylist.insertLast(40);  
mylist.insertLast(50);
```

```
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist);
```

```
mylist.insert(2, 99);
```

```
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist);
```

```
mylist.deleteFirst();
```

```
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist);
```

```
mylist.deleteLast();
```



```
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist);
```

```
mylist.delete(2);  
System.out.println("-----");  
System.out.println(mylist.size());  
System.out.println(mylist);
```

```
System.out.println("-----");
```

```
int x= mylist.search(55);  
System.out.println(x);
```

```
x= mylist.search(99);  
System.out.println(x);
```

```
x= mylist.search(30);  
System.out.println(x);
```

```
}
```

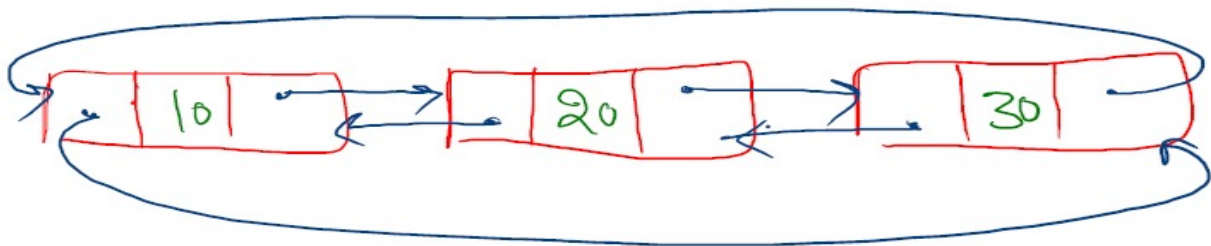
```
}
```



9.6. Circular Doubly Linked List

- ♦ Circular Doubly Linked List are same as Doubly Linked List without having node with NULL.
- ♦ Circular Doubly Linked List consists of a number of nodes without having node with NULL.
- ♦ Each Node has will have 3 parts
 - a) Address of Previous Node
 - b) Data
 - c) Address of Next Node
- ♦ Address of the Last node in the list is First Node Address

Ex:



- ♦ Following is a type declaration for the Node:

```
public class Node {  
  
    int data;  
    Node next;  
    Node prev;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
        this.prev=null;  
    }  
}
```

- ◆ Main Circular Doubly Linked List Operations
 - ✓ Traverse: Access the elements in the List
 - ✓ Count: returns the number of elements in the List
 - ✓ Find nth node from the end of the List
 - ✓ Insert: inserts an element into the List
 - ✓ Delete: removes the specified position element from the List

9.6.1. Traverse the Circular Doubly LinkedList in Forward Order

Lab34.java

```
package com.jlcindia.circular.doubly.linkedlist;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Traverse the Circular Doubly LinkedList in Forward Order
public class Lab34 {

    static void displayList(Node headNode) {

        if (headNode == null) {
            return;
        }

        Node currentNode = headNode;

        do {
            System.out.print(currentNode.data + "\t");
            currentNode = currentNode.next;
        } while (currentNode != headNode);

        System.out.println("\n");
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = new Node(10);  
    Node node2 = new Node(20);  
    Node node3 = new Node(30);  
    Node node4 = new Node(40);  
  
    head.next = node2;  
    head.prev=node4;  
  
    node2.next = node3;  
    node2.prev=head;  
  
    node3.next = node4;  
    node3.prev=node2;  
  
    node4.next = head;  
    node4.prev=node3;  
  
    displayList(head);  
  
}
```



9.6.2. Traverse the Circular Doubly LinkedList in Reverse Order

Lab35.java

```
package com.jlccindia.circular.doubly.linkedlist;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Traverse the Circular Doubly LinkedList in Reverse Order
public class Lab35 {

    static void displayList(Node headNode) {

        if (headNode == null) {
            return;
        }

        Node currentNode = headNode.prev; //

        do {
            System.out.print(currentNode.data + "\t");
            currentNode = currentNode.prev;
        } while (currentNode != headNode);

        System.out.println(headNode.data);

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);
        Node node4 = new Node(40);
```



```
        head.next = node2;
        head.prev = node4;

        node2.next = node3;
        node2.prev = head;

        node3.next = node4;
        node3.prev = node2;

        node4.next = head;
        node4.prev = node3;

        displayList(head);
    }
}
```

9.6.3. Count the Nodes in Circular Doubly LinkedList

Lab36.java

```
package com.jlcindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Count Nodes in Circular Doubly LinkedList
public class Lab36 {

    static int length(Node headNode) {

        if (headNode == null) {
            return 0;
        }

        int length=0;
        Node currentNode = headNode;
```



```
        do {
            length++;
            currentNode = currentNode.next;
        } while (currentNode != headNode);

        return length;
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);
        Node node4 = new Node(40);

        head.next = node2;
        head.prev=node4;

        node2.next = node3;
        node2.prev=head;

        node3.next = node4;
        node3.prev=node2;

        node4.next = head;
        node4.prev=node3;

        int x= length(head);
        System.out.println(x);
    }
}
```



9.6.4. Insert the Node at beginning of Circular Doubly LinkedList

Lab37.java

```
package com.jlccindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Insert the Node at beginning of Circular Doubly LinkedList
public class Lab37 {

    static Node insertFirst(Node headNode, int data) {

        Node temp = new Node(data);

        // 1.Empty List
        if (headNode == null) {
            headNode = temp;
            headNode.next = headNode;
            headNode.prev = headNode;
            //Head Changed
            return headNode;
        }

        // 2.Non-Empty List
        temp.next = headNode;
        temp.prev = headNode.prev;
        headNode.prev.next = temp;
        headNode.prev = temp;

        // Head Changed
        headNode = temp;

        return headNode;
    }
}
```



```
static void displayList(Node headNode) {

    if (headNode == null) {
        return;
    }

    Node currentNode = headNode;

    do {
        System.out.print(currentNode.data + "\t");
        currentNode = currentNode.next;
    } while (currentNode != headNode);

    System.out.println("\n");
}

public static void main(String[] args) {

    Node head= null;
    displayList(head);

    head = insertFirst(head,99);
    displayList(head);

    head = insertFirst(head,10);
    head = insertFirst(head,20);
    head = insertFirst(head,30);
    displayList(head);

}
}
// Time - O(1)
//Space - O(1)
```



9.6.5. Insert the Node at ending of Circular Doubly LinkedList

Lab38.java

```
package com.jlccindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Insert the Node at end of Circular Doubly LinkedList
public class Lab38 {

    static Node insertLast(Node headNode, int data) {

        Node temp = new Node(data);

        // 1.Empty List
        if (headNode == null) {
            headNode = temp;
            headNode.next = headNode;
            headNode.prev = headNode;
            //Head Changed
            return headNode;
        }

        // 2.Non-Empty List
        temp.next = headNode;
        temp.prev = headNode.prev;
        headNode.prev.next = temp;
        headNode.prev = temp;

        // Head Not Changed

        return headNode;
    }
}
```



```
static void displayList(Node headNode) {

    if (headNode == null) {
        return;
    }

    Node currentNode = headNode;

    do {
        System.out.print(currentNode.data + "\t");
        currentNode = currentNode.next;
    } while (currentNode != headNode);

    System.out.println("\n");
}

public static void main(String[] args) {

    Node head= null;
    displayList(head);

    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,99);
    displayList(head);

}
}
```

//Time - O(1)
//Space - O(1)

9.6.6. Insert the Node at given position of Circular Doubly LinkedList

Lab39.java

```
package com.jlccindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Insert the Node at given position of Circular Doubly LinkedList
public class Lab39 {

    static Node insert(Node headNode,int position, int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            return headNode;
        }

        //2. Position = 1
        if(position==1){
            return insertFirst(headNode, data);
        }

        //3.Position >=2
        // Traverse before to given position
        Node currentNode = headNode;
        for(int i=1;i<=position-2 && currentNode.next != headNode;i++) {
            currentNode=currentNode.next;
        }

        //Position is out of range
        if(currentNode.next==headNode) {
            return headNode;
        }
    }
}
```



```
//Position is in range
temp.next=currentNode.next;
temp.prev=currentNode;

currentNode.next.prev=temp;
currentNode.next = temp;

//HeadNode is Not Changed
return headNode;
}
static Node insertFirst(Node headNode, int data) {
    Refer Lab37
}
static void displayList(Node headNode) {
    Refer Lab37
}
public static void main(String[] args) {

    Node head = null;
    head = insertFirst(head,50);
    head = insertFirst(head,40);
    head = insertFirst(head,30);
    head = insertFirst(head,20);
    head = insertFirst(head,10);

    displayList(head);
    System.out.println("-----");

    head = insert(head,3,99);
    displayList(head);
}
}
// Time - O(n)
//Space - O(1)
```

9.6.7. Delete the First Node in Circular Doubly LinkedList

Lab40.java

```
package com.jlccindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Delete the First Node of Circular Doubly LinkedList

public class Lab40 {

    static Node deleteFirst(Node headNode) {

        //1. Empty List
        if(headNode==null) {
            return null;
        }

        //2. One Node List
        if(headNode.next==headNode) {

            headNode.next=null;
            headNode.prev=null;

            return null;
        }

        //3.List with 2 or more Nodes

        headNode.data=headNode.next.data;
        Node secondNode = headNode.next;

        headNode.next=secondNode.next;
        secondNode.next.prev=headNode;

        secondNode.next=null;
    }
}
```



```
        secondNode.prev=null;

        return headNode;
    }

    static Node insertFirst(Node headNode, int data) {
        Refer Lab38
    }
    static void displayList(Node headNode) {
        Refer Lab38
    }
    public static void main(String[] args) {

        Node head= null;

        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);
        displayList(head);

        head= deleteFirst(head);
        displayList(head);

        head= deleteFirst(head);
        displayList(head);

    }
}
// Time - O(1)
//Space - O(1)
```



9.6.8. Delete the Last Node in Circular Singly LinkedList

Lab41.java

```
package com.jlccindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Delete the Last Node of Circular Doubly LinkedList

public class Lab41 {

    static Node deleteLast(Node headNode) {

        //1. Empty List
        if(headNode==null) {
            return null;
        }

        //2. One Node List
        if(headNode.next==headNode) {

            headNode.next=null;
            headNode.prev=null;

            return null;
        }

        //3.List with 2 or more Nodes

        //Get Node before to Last Node
        Node currentNode = headNode.prev.prev;
        Node lastNode= currentNode.next;

        currentNode.next=headNode;
        headNode.prev=currentNode;
    }
}
```



```
        lastNode.next=null;
        lastNode.prev=null;

        return headNode;
    }
    static Node insertFirst(Node headNode, int data) {
        Refer Lab38
    }
    static void displayList(Node headNode) {
        Refer Lab38
    }
    public static void main(String[] args) {

        Node head= null;

        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);
        displayList(head);

        head= deleteLast(head);
        displayList(head);

        head= deleteLast(head);
        displayList(head);

    }
}
// Time - O(1)
//Space - O(1)
```



9.6.9. Delete the Node at given position in Circular Doubly LinkedList

Lab42.java

```
package com.jlcindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Delete the Node at given position in Circular Doubly LinkedList
public class Lab42 {
    static Node delete(Node headNode, int position) {

        // 1. Empty List
        if (headNode == null) {
            return headNode;
        }

        // 2. One Node List
        if (headNode.next == headNode) {
            return headNode;
        }

        // 3. List with 2 or more Nodes
        // Traverse before the Nth Position
        Node currentNode = headNode;
        for (int i = 1; i <= position - 2 && currentNode.next != headNode; i++) {
            currentNode = currentNode.next;
        }

        // When Position is out of range
        if (currentNode.next == headNode) {
            return headNode;
        }

        // When Position is in range
        Node temp = currentNode.next.next;
        Node nodeToDelete = currentNode.next;
```



```
        currentNode.next = temp;
        temp.prev= currentNode;

        //Free the Memory
        nodeToDelete.next=null;
        nodeToDelete.prev=null;

        return headNode;
    }
    static Node insertFirst(Node headNode, int data) {
        Refer Lab38
    }
    static void displayList(Node headNode) {
        Refer Lab38
    }
    public static void main(String[] args) {

        Node head = null;

        head = insertLast(head, 10);
        head = insertLast(head, 20);
        head = insertLast(head, 30);
        head = insertLast(head, 40);
        head = insertLast(head, 50);
        displayList(head);

        head = delete(head,4);
        displayList(head);
        head = delete(head,3);
        displayList(head);
    }
}
// Time - O(n)
//Space - O(1)
```



9.6.10. Search the Element in Circular Doubly LinkedList

Lab43.java

```
package com.jlcindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Search the Element in Circular Doubly LinkedList

public class Lab43 {

    static int search(Node headNode, int element) {

        if (headNode == null) {
            return -1;
        }

        int position = 1;
        Node currentNode = headNode;

        do {
            if (currentNode.data == element) {
                return position;
            } else {
                position++;
                currentNode = currentNode.next;
            }
        } while (currentNode != headNode);

        return -1;
    }
}
```



```
static Node insertFirst(Node headNode, int data) {
```

Refer Lab38

```
}
```

```
static void displayList(Node headNode) {
```

Refer Lab38

```
}
```

```
public static void main(String[] args) {
```

```
    Node head = null;
```

```
    head = insertLast(head, 10);
```

```
    head = insertLast(head, 20);
```

```
    head = insertLast(head, 30);
```

```
    head = insertLast(head, 40);
```

```
    head = insertLast(head, 50);
```

```
    displayList(head);
```

```
    System.out.println("-----");
```

```
    int x = search(head, 99);
```

```
    System.out.println(x);
```

```
    System.out.println("-----");
```

```
    x = search(head, 30);
```

```
    System.out.println(x);
```

```
    System.out.println("-----");
```

```
    x = search(head, 50);
```

```
    System.out.println(x);
```

```
}
```

```
}
```

```
// Time - O(n)
```

```
//Space - O(1)
```



9.6.11. Design MyCircularDoublyLinkedList

Lab44.java

```
package com.jlcindia.circular.doubly.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Design MyCircularDoublyLinkedList

public class MyCircularDoublyLinkedList {

    Node headNode = null;
    int nodeCount = 0;

    public MyCircularDoublyLinkedList() {
    }

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = this.headNode;
        do {
            str = str + "" + currentNode.data + " , ";
            currentNode = currentNode.next;
        } while (currentNode != headNode);

        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }
}
```



```
public boolean isEmpty() {
    return nodeCount == 0;
}

public void clear() {

    Node currentNode = headNode;
    do {
        Node temp = currentNode.next;

        currentNode.next = null;
        currentNode = temp;
    } while (currentNode != headNode);

    this.headNode = null;
    this.nodeCount = 0;
}

public int size() {
    return nodeCount;
}

public void insertFirst(int data) {

    Node temp = new Node(data);

    // 1.Empty List
    if (this.headNode == null) {
        this.headNode = temp;
        this.headNode.next = this.headNode;
        this.headNode.prev = this.headNode;
        // Head Changed
        nodeCount++;
        return;
    }
}
```



```
// 2.Non-Empty List
temp.next = headNode;
temp.prev = headNode.prev;
headNode.prev.next = temp;
headNode.prev = temp;

// Head Changed
this.headNode = temp;
nodeCount++;

}

public void insertLast(int data) {

    Node temp = new Node(data);

    // 1.Empty List
    if (this.headNode == null) {
        this.headNode = temp;
        this.headNode.next = this.headNode;
        this.headNode.prev = this.headNode;
        // Head Changed
        nodeCount++;
        return;
    }

    // 2.Non-Empty List
    temp.next = headNode;
    temp.prev = headNode.prev;
    headNode.prev.next = temp;
    headNode.prev = temp;

    // Head Not Changed
    nodeCount++;

}
```



```
public void insert(int position, int data) {

    Node temp = new Node(data);

    // 1.Empty List
    if (headNode == null) {
        return;
    }

    // 2. Position = 1
    if (position == 1) {
        insertFirst(data);
    }

    // 3.Position >=2
    // Traverse before to given position
    Node currentNode = headNode;
    for (int i = 1; i <= position - 2 && currentNode.next != headNode; i++) {
        currentNode = currentNode.next;
    }

    // Position is out of range
    if (currentNode.next == headNode) {
        return;
    }

    // Position is in range
    temp.next = currentNode.next;
    temp.prev = currentNode;

    currentNode.next.prev = temp;
    currentNode.next = temp;

    // HeadNode is Not Changed
    nodeCount++;
}
```



```
public void deleteFirst() {  
  
    // 1. Empty List  
    if (headNode == null) {  
        return;  
    }  
  
    // 2. One Node List  
    if (headNode.next == headNode) {  
        headNode.next = null;  
        headNode.prev = null;  
        nodeCount--;  
        return;  
    }  
  
    // 3. List with 2 or more Nodes  
  
    headNode.data = headNode.next.data;  
    Node secondNode = headNode.next;  
  
    headNode.next = secondNode.next;  
    secondNode.next.prev = headNode;  
  
    secondNode.next = null;  
    secondNode.prev = null;  
  
    // Head Not Changed  
    nodeCount--;  
}
```



```
public void deleteLast() {  
  
    // 1. Empty List  
    if (headNode == null) {  
        return;  
    }  
  
    // 2. One Node List  
    if (headNode.next == headNode) {  
  
        headNode.next = null;  
        headNode.prev = null;  
  
        nodeCount--;  
        return;  
    }  
  
    // 3. List with 2 or more Nodes  
  
    // Get Node before to Last Node  
    Node currentNode = headNode.prev.prev;  
    Node lastNode = currentNode.next;  
  
    currentNode.next = headNode;  
    headNode.prev = currentNode;  
  
    lastNode.next = null;  
    lastNode.prev = null;  
  
    // Head Not Changed  
    nodeCount--;  
  
}
```



```
public void delete(int position) {  
    // 1. Empty List  
    if (headNode == null) {  
        return;  
    }  
    // 2. One Node List  
    if (headNode.next == headNode) {  
        return;  
    }  
  
    // 3. List with 2 or more Nodes  
    // Traverse before the Nth Position  
    Node currentNode = headNode;  
    for (int i = 1; i <= position - 2 && currentNode.next != headNode; i++) {  
        currentNode = currentNode.next;  
    }  
  
    // When Position is out of range  
    if (currentNode.next == headNode) {  
        return;  
    }  
  
    // When Position is in range  
    Node temp = currentNode.next.next;  
    Node nodeToDelete = currentNode.next;  
  
    currentNode.next = temp;  
    temp.prev = currentNode;  
  
    // Free the Memory  
    nodeToDelete.next = null;  
    nodeToDelete.prev = null;  
  
    // Head Not Changed  
    nodeCount--;  
}
```



```
public int search(int element) {

    if (headNode == null) {
        return -1;
    }

    int position = 1;
    Node currentNode = headNode;

    do {
        if (currentNode.data == element) {
            return position;
        } else {
            position++;
            currentNode = currentNode.next;
        }
    } while (currentNode != headNode);

    return -1;
}

// Test MyCircularDoublyLinkedList

public class Lab44{
    public static void main(String[] args) {

        MyCircularDoublyLinkedList mylist = new MyCircularDoublyLinkedList();

        System.out.println(mylist.size());
        System.out.println(mylist.isEmpty());
        System.out.println(mylist);
    }
}
```



```
mylist.insertFirst(99);
mylist.insertFirst(88);

System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist.isEmpty());
System.out.println(mylist);

mylist.clear();

System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist.isEmpty());
System.out.println(mylist);

mylist.insertLast(10);
mylist.insertLast(20);
mylist.insertLast(30);
mylist.insertLast(40);
mylist.insertLast(50);

System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist);

mylist.insert(3, 99);

System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist);

mylist.deleteFirst();

System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist);
```



```
mylist.deleteLast();

System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist);

mylist.delete(2);
System.out.println("-----");
System.out.println(mylist.size());
System.out.println(mylist);

System.out.println("-----");

int x= mylist.search(55);
System.out.println(x);

x= mylist.search(99);
System.out.println(x);

x= mylist.search(30);
System.out.println(x);

}
}
```